Project Check-in 2 — CS-665

Recipe Finder: Normalization and Architectural Evolution

Abhiram Golkonda · B855Q929

---

1. Schema Evolution and ER Modeling

1.1 Why the Check-in 1 schema has to grow

Check-in 1 used two tables, **Recipes** and **Ingredients**. That was enough to answer "which recipes use chicken?", but the app's real purpose is *"what can I cook with what I already have?"* That needs more than the first design can give:

- **A pantry.** The app has to know which ingredients the user currently owns, independent of any recipe.
- **Ingredients as their own thing.** "Onion" should exist once, no matter how many recipes use it.
- **Browsable recipes.** Users want to filter by category (Breakfast, Lunch, Dinner).
- **Reliable quantities.** `'500 g'` and `'2 cloves'` stored as one text value cannot be scaled, compared, or validated.

1.2 Final entities

Entity	Purpose	Primary key
`Categories`	Meal categories	`category_id`
`Recipes`	A saved recipe	`recipe_id`
`Ingredients`	One row per distinct ingredient	`ingredient_id`
`Units`	Measurement units (g, tbsp, clove, ...)	`unit_id`
`Recipe_Ingredients`	Which ingredient a recipe uses, how much, in what unit	(`recipe_id` , `ingredient_id`)
`Pantry`	Ingredients the user currently has	`ingredient_id`

1.3 Relationships

Relationship	Cardinality	How it is implemented
Category → Recipe	one-to-many	`Recipes.category_id` (FK)
Recipe ↔ Ingredient	**many-to-many**	junction table `Recipe_Ingredients`
Unit → Recipe_Ingredients	one-to-many	`Recipe_Ingredients.unit_id` (FK)
Ingredient → Pantry	one-to-zero-or-one	`Pantry.ingredient_id` is both PK and FK

The most important change from Check-in 1 is the Recipe ↔ Ingredient relationship. In Check-in 1 I modeled it as one-to-many (each ingredient row "belonged" to one recipe). In reality *Onion* is used by both Chicken Curry and Veggie Omelette, so the relationship is many-to-many and needs a junction table.

---

2. Step-by-Step Normalization

I use the **same four recipes and the same facts** at every step so it is easy to check that nothing is lost.

2.0 Starting point: Unnormalized form (UNF)

This is everything the expanded app wants to store, written as one big record per recipe:

recipe_id	recipe_name	category_id	category_name	instructions	ingredients (name, quantity)
1	Chicken Curry	1	Dinner	Saute onion, add chicken and curry powder, simmer.	Chicken 500 g; Onion 1; Curry Powder 2 tbsp
2	Tomato Pasta	1	Dinner	Boil pasta, cook tomato and garlic, combine.	Pasta 200 g; Tomato 3; Garlic 2 cloves
3	Veggie Omelette	2	Breakfast	Beat eggs, add onion and tomato, cook in a pan.	Eggs 3; Onion 1; Tomato 1
4	Chicken Salad	3	Lunch	Chop cooked chicken, mix with lettuce.	Chicken 200 g; Lettuce 1 head

Problem: the last column holds a *list* of values per row (a repeating group). That violates 1NF.

2.1 First Normal Form (1NF)

Rule: every cell holds a single atomic value, there are no repeating groups, and each row is uniquely identified by a key.

Changes made

1. Each ingredient gets its own row (removes the repeating group).
2. `quantity` is split into atomic `amount` and `unit` ("2 tbsp" becomes `2` + `tbsp`). Items counted as whole things (onion, eggs) use the unit `whole`.
3. The key becomes the composite (`recipe_id`, `ingredient_name`).

Table `RecipeData_1NF` — key (`recipe_id`, `ingredient_name`). The long `instructions` text is repeated on every row but omitted below for width.

recipe_id	ingredient_name	amount	unit	recipe_name	category_id	category_name
1	Chicken	500	g	Chicken Curry	1	Dinner
1	Onion	1	whole	Chicken Curry	1	Dinner
1	Curry Powder	2	tbsp	Chicken Curry	1	Dinner
2	Pasta	200	g	Tomato Pasta	1	Dinner
2	Tomato	3	whole	Tomato Pasta	1	Dinner
2	Garlic	2	clove	Tomato Pasta	1	Dinner
3	Eggs	3	whole	Veggie Omelette	2	Breakfast
3	Onion	1	whole	Veggie Omelette	2	Breakfast
3	Tomato	1	whole	Veggie Omelette	2	Breakfast
4	Chicken	200	g	Chicken Salad	3	Lunch
4	Lettuce	1	head	Chicken Salad	3	Lunch

Check: 11 rows, matching the 11 ingredient entries in Check-in 1. All data is still here. But there is a lot of repetition ("Chicken Curry / Dinner" three times), which is the sign of the next problem.

2.2 Second Normal Form (2NF)

Rule: be in 1NF, and every non-key attribute must depend on the **whole** composite key, not just part of it (no partial dependencies).

Functional dependencies in `RecipeData_1NF` (key = `recipe_id`, `ingredient_name`)

- `recipe_id` → `recipe_name`, `category_id`, `category_name`, `instructions` (**partial** dependency: depends on only half of the key)

- (`recipe_id` , `ingredient_name`) → `amount` , `unit` (full dependency: the amount depends on the *pair*)

The recipe attributes violate 2NF, so I split along the partial dependency.

Table `Recipes_2NF` — key `recipe_id`

recipe_id	recipe_name	category_id	category_name	instructions
1	Chicken Curry	1	Dinner	Saute onion, add chicken and curry powder, simmer.
2	Tomato Pasta	1	Dinner	Boil pasta, cook tomato and garlic, combine.
3	Veggie Omelette	2	Breakfast	Beat eggs, add onion and tomato, cook in a pan.
4	Chicken Salad	3	Lunch	Chop cooked chicken, mix with lettuce.

Table `RecipeIngredients_2NF` — key (`recipe_id` , `ingredient_name`)

recipe_id	ingredient_name	amount	unit
1	Chicken	500	g
1	Onion	1	whole
1	Curry Powder	2	tbsp
2	Pasta	200	g
2	Tomato	3	whole
2	Garlic	2	clove
3	Eggs	3	whole
3	Onion	1	whole
3	Tomato	1	whole
4	Chicken	200	g
4	Lettuce	1	head

Check: joining the two tables on `recipe_id` reproduces all 11 rows of `RecipeData_1NF` . Nothing was lost, and each recipe's name, category, and instructions are now stored once.

2.3 Third Normal Form (3NF)

Rule: be in 2NF, and no non-key attribute may depend on another non-key attribute (no transitive dependencies).

Transitive dependency found in `Recipes_2NF`:

`recipe_id` → `category_id` → `category_name`

`category_name` is determined by `category_id`, not directly by the recipe. "Dinner" is still stored twice (recipes 1 and 2). I move it into its own table.

Table `Categories` — key `category_id`

category_id	category_name
1	Dinner
2	Breakfast
3	Lunch

Table `Recipes` — key `recipe_id`, FK `category_id`

recipe_id	recipe_name	category_id	instructions
1	Chicken Curry	1	Saute onion, add chicken and curry powder, simmer.
2	Tomato Pasta	1	Boil pasta, cook tomato and garlic, combine.
3	Veggie Omelette	2	Beat eggs, add onion and tomato, cook in a pan.
4	Chicken Salad	3	Chop cooked chicken, mix with lettuce.

Further decomposition for the many-to-many relationship and the pantry. In `RecipeIngredients_2NF`, `ingredient_name` is repeated text (Chicken, Onion, and Tomato each appear twice). Strictly speaking this table already satisfies 3NF. I still promote ingredients to their own entity, because the new Pantry feature needs a single authoritative list of ingredients, and a typo such as "Tomatoe" in one recipe would otherwise silently break matching. I do the same for units, so that "g" and "grams" cannot both exist.

Table `Ingredients`

ingredient_id	ingredient_name
1	Chicken
2	Onion
3	Curry Powder
4	Pasta
5	Tomato
6	Garlic
7	Eggs
8	Lettuce

Table `Units`

unit_id	unit_name
1	g
2	tbsp
3	clove
4	head
5	whole

Table `Recipe_Ingredients` — key (`recipe_id` , `ingredient_id`)

recipe_id	ingredient_id	amount	unit_id
1	1 (Chicken)	500	1 (g)
1	2 (Onion)	1	5 (whole)
1	3 (Curry Powder)	2	2 (tbsp)
2	4 (Pasta)	200	1 (g)
2	5 (Tomato)	3	5 (whole)
2	6 (Garlic)	2	3 (clove)
3	7 (Eggs)	3	5 (whole)

---

recipe_id	ingredient_id	amount	unit_id
3	2 (Onion)	1	5 (whole)
3	5 (Tomato)	1	5 (whole)
4	1 (Chicken)	200	1 (g)
4	8 (Lettuce)	1	4 (head)

Table `Pantry` (new for the app; example contents for a user who has these on hand)

ingredient_id	date_added
1 (Chicken)	2026-10-20
2 (Onion)	2026-10-20
5 (Tomato)	2026-10-22
6 (Garlic)	2026-10-22
7 (Eggs)	2026-10-25

2.4 Proof that no information was lost

The 11-row 1NF table can be rebuilt exactly from the 3NF tables:

```
SELECT r.recipe_id, i.ingredient_name, ri.amount, u.unit_name,
       r.recipe_name, c.category_id, c.category_name
FROM Recipes r
JOIN Categories c         ON c.category_id   = r.category_id
JOIN Recipe_Ingredients ri ON ri.recipe_id    = r.recipe_id
JOIN Ingredients i        ON i.ingredient_id = ri.ingredient_id
JOIN Units u              ON u.unit_id       = ri.unit_id
ORDER BY r.recipe_id, i.ingredient_id;
```

Expected result: the same 11 rows as `RecipeData_1NF` (for example, 1 / Chicken / 500 / g / Chicken Curry / 1 / Dinner). This is a **lossless-join decomposition**: every split table shares a key with the rest, so joining recovers the original facts.

2.5 Final 3NF schema (SQL)

```
CREATE TABLE Categories (
    category_id   INT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE
);
```

```
CREATE TABLE Units (
    unit_id   INT PRIMARY KEY,
    unit_name VARCHAR(30) NOT NULL UNIQUE
);

CREATE TABLE Recipes (
    recipe_id    INT PRIMARY KEY,
    recipe_name  VARCHAR(255) NOT NULL,
    category_id  INT NOT NULL,
    instructions TEXT,
    FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);

CREATE TABLE Ingredients (
    ingredient_id   INT PRIMARY KEY,
    ingredient_name VARCHAR(255) NOT NULL UNIQUE
);

CREATE TABLE Recipe_Ingredients (
    recipe_id     INT NOT NULL,
    ingredient_id INT NOT NULL,
    amount        DECIMAL(8,2) NOT NULL,
    unit_id       INT NOT NULL,
    PRIMARY KEY (recipe_id, ingredient_id),
    FOREIGN KEY (recipe_id)     REFERENCES Recipes(recipe_id) ON DELETE
CASCADE,
    FOREIGN KEY (ingredient_id) REFERENCES Ingredients(ingredient_id),
    FOREIGN KEY (unit_id)       REFERENCES Units(unit_id)
);

CREATE TABLE Pantry (
    ingredient_id INT PRIMARY KEY,
    date_added    DATE,
    FOREIGN KEY (ingredient_id) REFERENCES Ingredients(ingredient_id)
);
```

Integrity decisions:

- `UNIQUE` on `ingredient_name`, `category_name`, and `unit_name` blocks duplicate or misspelled variants.
- The composite primary key on `Recipe_Ingredients` prevents listing the same ingredient twice in one recipe.
- `ON DELETE CASCADE` on `recipe_id` means deleting a recipe removes only its link rows, never the ingredients themselves.

- `Pantry.ingredient_id` is a primary key, so an ingredient can be in the pantry at most once.

2.6 Core app query on the new schema

The Check-in 1 query "recipes that use Chicken" becomes:

```
SELECT r.recipe_name
FROM Recipes r
JOIN Recipe_Ingredients ri ON ri.recipe_id    = r.recipe_id
JOIN Ingredients i         ON i.ingredient_id = ri.ingredient_id
WHERE i.ingredient_name = 'Chicken';
```

Expected result: Chicken Curry, Chicken Salad (unchanged).

The new headline feature, *"what can I cook from my pantry?"*, ranks recipes by how many ingredients are missing:

```
SELECT r.recipe_name,
       COUNT(*) AS total_ingredients,
       SUM(CASE WHEN p.ingredient_id IS NULL THEN 1 ELSE 0 END) AS missing
FROM Recipes r
JOIN Recipe_Ingredients ri ON ri.recipe_id = r.recipe_id
LEFT JOIN Pantry p         ON p.ingredient_id = ri.ingredient_id
GROUP BY r.recipe_id, r.recipe_name
ORDER BY missing ASC;
```

Expected result with the sample pantry above:

recipe_name	total_ingredients	missing
Veggie Omelette	3	0
Chicken Curry	3	1 (Curry Powder)
Tomato Pasta	3	1 (Pasta)
Chicken Salad	2	1 (Lettuce)

Veggie Omelette is fully cookable. The other three are each missing exactly one ingredient, which is useful for a shopping suggestion.

---

3. The Problem-Solving Journey

3.1 Functional dependencies

In the Check-in 1 design

- `Recipes`: `recipe_id` → `recipe_name`, `instructions`
- `Ingredients`: `ingredient_id` → `recipe_id`, `ingredient_name`, `quantity`

Both are keyed correctly, so the tables look fine on paper. The problems are in what the design *cannot express*: it implicitly treats the pair (ingredient, recipe) as a single entity and repeats ingredient names for every use.

In the expanded "everything in one table" form (the 1NF table)

Dependency	Type
`recipe_id` → `recipe_name`, `category_id`, `instructions`	Partial (violates 2NF)
`category_id` → `category_name`	Makes `recipe_id` → `category_name` transitive (violates 3NF)
(`recipe_id`, `ingredient_id`) → `amount`, `unit_id`	Full dependency (correct)
`ingredient_id` → `ingredient_name`	Ingredient facts independent of recipe
`unit_id` → `unit_name`	Unit facts independent of recipe

In the final schema, every determinant is a candidate key of its own table: `recipe_id`, `category_id`, `ingredient_id`, `unit_id`, and (`recipe_id`, `ingredient_id`). That is the practical test I used for 3NF (and in fact the stronger BCNF holds too).

3.2 Anomalies I found in the early designs

A. Update anomaly (Check-in 1 `Ingredients` table). Chicken appears in rows 1 and 10, Onion in rows 2 and 8, and Tomato in rows 5 and 9. If I rename "Tomato" to "Tomatoes" and update only row 5, then row 9 still says "Tomato". The ingredient search silently returns the wrong set of recipes, and the database cannot tell me which spelling is right.

B. Insertion anomaly (Check-in 1). `recipe_id` is `NOT NULL` in `Ingredients`, so I cannot record that the user owns **Rice** unless I invent a recipe that uses it. The pantry feature is impossible in the old design. The same issue appeared in the big table: I could not add a new category like "Dessert" until a recipe belonged to it.

C. Deletion anomaly (Check-in 1). Lettuce appears only in row 11 (Chicken Salad). Deleting the Chicken Salad recipe deletes the only record that Lettuce exists. In the big table, deleting Veggie Omelette would also erase the only evidence that the **Breakfast** category exists.

D. Redundancy in the 1NF table. "Chicken Curry / 1 / Dinner" is stored three times and the recipe instructions are copied onto every ingredient row. Changing the category of Chicken Curry would require editing three rows, and missing one would leave the recipe in two categories at once.

3.3 How the final 3NF design resolves each one

Anomaly	Resolution in the final design
Update (A)	`Ingredients.ingredient_name` is stored in exactly one row per ingredient. Renaming Tomato is a single `UPDATE` and every recipe sees it through the join.
Insertion (B)	Rice can be added to `Ingredients` and `Pantry` with no recipe. A `Dessert` category can exist with zero recipes.
Deletion (C)	Deleting Chicken Salad removes only its `Recipe_Ingredients` link rows (via `ON DELETE CASCADE`). Lettuce stays in `Ingredients`, and Breakfast stays in `Categories`.
Redundancy (D)	Category, recipe, and instructions are each stored once. Changing a category is one `UPDATE` on one row.

3.4 Reflection on the troubleshooting process

Where I started. I began by testing the Check-in 1 schema with "what if" operations instead of just reading it. I tried to rename an ingredient, insert an ingredient with no recipe, and delete a recipe, and watched what each would do to the data. Doing it on paper with my sample rows made the three anomalies in 3.2 concrete rather than abstract definitions.

First idea that did not work. My first fix was to pull `ingredient_name` into its own `Ingredients` table and keep `quantity` there. That broke immediately: the *amount* is not a property of Chicken, it is a property of Chicken **in Chicken Curry** (500 g) versus Chicken **in Chicken Salad** (200 g). Asking "what does `quantity` depend on?" showed it

depends on the *pair* (recipe, ingredient). That is how I realized the relationship was really many-to-many and needed a junction table. Writing the functional dependency down as (`recipe_id` , `ingredient_id`) → `amount` , `unit_id` is what finally made it clear.

Second dead end. I considered putting a default unit on each ingredient (Chicken → g). But Garlic appears as "2 cloves" here and could be "10 g" in another recipe, so the unit also depends on the pair and belongs in the junction table. I made `Units` its own lookup table so that spellings stay consistent.

Alternatives I rejected.

- Storing ingredients as a comma-separated list in `Recipes` . It looks simple, but it breaks 1NF and makes "search by ingredient" depend on text matching instead of joins.
- Keeping `category_name` as plain text in `Recipes` . It would be less work, but it creates the transitive dependency and the "Dinner" versus "dinner" problem.

How I checked my work. After each decomposition step, I joined the pieces back together and compared against the 11-row table to confirm nothing was lost, and then re-ran the Chicken query to confirm the app's core feature still returns Chicken Curry and Chicken Salad.

What I would do next. Check-in 3 can add indexes on `Recipe_Ingredients.ingredient_id` and `Ingredients.ingredient_name` , since the pantry query joins on them constantly, and consider whether numeric `amount` plus `unit` handles fractions like "1/2 tsp" well enough.